Chuleta de la UT4 — Spring Boot: capas, REST y JDBC

1 · Arranque y dependencias

```
@SpringBootApplication          // = @Configuration +
@EnableAutoConfiguration + @ComponentScan
@EnableCaching                  // sin esto, @Cacheable NO
hace nada
public class App {
    public static void main(String[] args)
{ SpringApplication.run(App.class, args); }
}
```

AVISO La clase va **en el paquete raíz**: `@ComponentScan` escanea desde ahí hacia abajo.

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.5.6</version>
</parent>
<properties><java.version>25</java.version></properties>
```

Starter	Trae
`spring-boot-starter-web`	Tomcat, `@RestController`, Jackson
`spring-boot-starter-validation`	`@Valid`, `@NotBlank`…
`spring-boot-starter-cache`	`@Cacheable`, `@CacheEvict`
`spring-boot-starter-jdbc`	`DataSource`, HikariCP, `JdbcClient`, `@Transactional`
`com.h2database:h2`	La base de datos *(scope `runtime`)*
`spring-boot-starter-test`	JUnit 5, Mockito, AssertJ, `MockMvc`

AVISO `starter-jdbc`, **NO** `starter-data-jpa`: en la UT4 el SQL lo escribes tú. JPA es la UT5. **AVISO** El `spring-boot-maven-plugin` es el que hace el `.jar` ejecutable.

```
./mvnw spring-boot:run
./mvnw spring-boot:run -Dspring-boot.run.profiles=dev
./mvnw clean package && java -jar target/app-1.0.0.jar
./mvnw spring-boot:run -Dspring-boot.run.arguments=--debug
# ver autoconfiguración
```

2 · Componentes e inyección

Anotación	Capa	Nota
`@RestController`	Web	`= @Controller + @ResponseBody`
`@Service`	Negocio	
`@Repository`	Datos	Traduce `SQLException` → `DataAccessException`
`@Component`	Genérica	Mappers, `RowMapper`, utilidades
`@Configuration` + `@Bean`	Config	Para clases que **no son tuyas**

```
@RestController
public class C {
    private final S servicio;                          //
final
    public C(S servicio) { this.servicio = servicio; }   //
constructor, sin @Autowired
}
```

Scopes: `singleton` (defecto) · `prototype` · `request` · `session`

AVISO Singleton = **una instancia para toda la app, en paralelo** → los servicios **no guardan estado**.

3 · Rutas, peticiones y respuestas

```
@RestController
@RequestMapping("${api.path:/api}/${api.version:v1}/funkos")
public class FunkosRestController {
    @GetMapping                 // GET  /api/v1/funkos
    @GetMapping("/{id}")        // GET  /api/v1/funkos/1
    @PostMapping                // POST
    @PutMapping("/{id}")        // PUT    reemplaza TODO
    @PatchMapping("/{id}")      // PATCH  cambia solo lo
enviado
    @DeleteMapping("/{id}")     // DELETE
}
```

Anotación	De dónde	Ejemplo
`@PathVariable`	Ruta	`/funkos/42`
`@RequestParam`	Query	`?categoria=disney`
`@RequestBody`	Cuerpo JSON	POST / PUT / PATCH
`@RequestHeader`	Cabecera	`Accept`, `Authorization`

```
@RequestParam(required = false) String q
@RequestParam(defaultValue = "0") int page
@RequestParam Optional<String> categoria            // ←
preferido
@RequestParam List<String> tags                     // ?
tags=a&tags=b
```

Criterio: identifica el recurso → ruta. Filtra, ordena o pagina → query. **Y la URL nombra el recurso, nunca la acción:** `/funkos/crear` está mal; el verbo HTTP ya lo dice.

```
ResponseEntity.ok(x);
200
ResponseEntity.created(URI.create("/api/v1/x/" +
id)).body(x);   // 201 + Location
ResponseEntity.noContent().build();
204
ResponseEntity.badRequest().body(e);
400
ResponseEntity.notFound().build();
404
ResponseEntity.status(HttpStatus.CONFLICT).body(e);
409
```

	Éxito	No existe	Datos malos	Choca
GET lista	200	— (200 vacía)	—	—
GET uno	200	**404**	—	—
POST	**201** + `Location`	—	400	**409**
PUT / PATCH	200	**404**	400	409
DELETE	**204**	**404**	—	409

AVISO Lista vacía = **200**, no 404 · DELETE de un id inexistente = **404**, no 204 **AVISO** Sin `Content-Type: application/json` en el POST → **415**

4 · DTOs

```
// RESPUESTA: lo que sale
public record FunkoResponse(Long id, String nombre, BigDecimal
precio,
                            LocalDateTime createdAt) {}

// CREAR (POST y PUT): SIN id ni fechas, con los obligatorios
public record FunkoCreateRequest(
        @NotBlank @Size(max = 100) String nombre,
        @NotNull @DecimalMin("0.0") @Digits(integer = 8,
fraction = 2) BigDecimal precio,
        @NotBlank @Pattern(regexp = "DISNEY|MARVEL|ANIME|
OTROS") String categoria) {}

// ACTUALIZAR (PATCH): sin @NotNull/@NotBlank, solo reglas de
formato
public record FunkoUpdateRequest(
        @Size(max = 100) String nombre,
        @DecimalMin("0.0") BigDecimal precio,
        @Pattern(regexp = "DISNEY|MARVEL|ANIME|OTROS") String
categoria) {}
```

AVISO Nunca devuelvas el modelo. Filtra datos y desacopla el contrato de la tabla. **AVISO** El DTO de creación **sin** `id`: si lo lleva, un cliente puede sobrescribir otro registro. **AVISO Nunca aceptes del cliente lo que puedas calcular**: total, precio congelado, estado inicial, código correlativo.

5 · Validación

```
@PostMapping
public ResponseEntity<R> crear(@Valid @RequestBody
CreateRequest dto) { … }
//                              ^^^^^^ sin esto NO SE VALIDA
NADA, y no avisa
```

`@NotNull`	No nulo → números, fechas, booleanos
`@NotBlank`	No nulo + no solo espacios → `String`
`@NotEmpty`	No nulo + `size > 0` → **colecciones**
`@Size(min=, max=)`	Longitud o tamaño
`@Min` `@Max`	Enteros
`@DecimalMin` `@DecimalMax` `@Digits(integer=, fraction=)`	`BigDecimal`
`@Positive` `@PositiveOrZero` `@Negative` `@NegativeOrZero`	Numéricos
`@Email` `@Pattern(regexp=)`	Texto
`@Past` `@PastOrPresent` `@Future` `@FutureOrPresent`	Fechas
`@AssertTrue` `@AssertFalse`	Validación **de clase**: compara dos campos
`@Valid`	**En cascada**: objetos y listas anidadas

```
record PedidoRequest(
    @NotNull Long clienteId,
    @NotEmpty @Valid List<LineaRequest> lineas) {}   // ← sin
@Valid, las líneas NO se validan
```

```
// Validación de clase, para reglas que comparan dos campos
record ReservaRequest(@NotNull LocalTime desde, @NotNull
LocalTime hasta) {
    @AssertTrue(message = "La hora de fin debe ser posterior")
    public boolean isRangoValido() {
        if (desde == null || hasta == null) return true;   //
de eso ya se queja @NotNull
        return hasta.isAfter(desde);
    }
}
```

Tres niveles, y se ponen los tres:

	Dónde	Qué comprueba	Código
Formato	DTO, anotaciones	Un campo aislado	**400**
Negocio	**Servicio**	Necesita consultar la BD u otros campos	**409**
Integridad	**Tabla**: `UNIQUE`, `CHECK`, `FK`	La única que no se puede saltar	409 vía handler

6 · Excepciones

```
public abstract class FunkoException extends RuntimeException
{
    protected FunkoException(String m) { super(m); }
}

@ResponseStatus(HttpStatus.NOT_FOUND)
public class FunkoNotFoundException extends FunkoException {
    public FunkoNotFoundException(Long id) { super("No existe
el funko con id: " + id); }
}
@ResponseStatus(HttpStatus.CONFLICT)    public class
FunkoConflictException   extends FunkoException { … }
@ResponseStatus(HttpStatus.BAD_REQUEST) public class
FunkoBadRequestException extends FunkoException { … }
```

```
// En el servicio: lanza, SIN mencionar HTTP
repositorio.findById(id).orElseThrow(() -> new
FunkoNotFoundException(id));
```

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ResponseStatus(HttpStatus.BAD_REQUEST)
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public Map<String, Object>
validacion(MethodArgumentNotValidException ex) {
        Map<String, String> campos = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors()
          .forEach(e -> campos.put(e.getField(),
e.getDefaultMessage()));
        return Map.of("status", 400, "error", "Datos
inválidos", "campos", campos);
    }


@ExceptionHandler(HttpMessageNotReadableException.class)
JSON roto      → 400

@ExceptionHandler(MethodArgumentTypeMismatchException.class)
funkos/abc    → 400

@ExceptionHandler(DataIntegrityViolationException.class)
UNIQUE, FK, CHECK → 409
}
```

AVISO Las excepciones **con** `@ResponseStatus` **no necesitan handler**: Spring las traduce solas. **AVISO Nunca** `ResponseStatusException` **en el servicio**: mete HTTP en la lógica y el servicio deja de poder reutilizarse (UT6).

7 · Caché

```
@Service
@CacheConfig(cacheNames = {"funkos"})
public class S {
    @Cacheable(key = "#id")        public R findById(Long id)
{ … }
    @CacheEvict(allEntries = true) public R save(C dto)
{ … }   // y update, replace, delete
    @CachePut(key = "#result.id")  public R actualizar(…)
{ … }
}
```

AVISO Sin `@EnableCaching` en la clase principal, **no hace nada**. **AVISO Los cuatro** métodos de escritura necesitan `@CacheEvict(allEntries = true)`, o `findAll` devuelve datos viejos. **AVISO** Una llamada **interna** (`this.findById(...)`) **no**

pasa por la caché: el proxy solo intercepta lo que entra de fuera. Igual que `@Transactional`.

8 · H2 y JDBC con `JdbcClient`

```
### H2 en FICHERO: los datos sobreviven
spring.datasource.url=${DB_URL:jdbc:h2:file:./datos/
funkos;AUTO_SERVER=TRUE}
spring.datasource.username=${DB_USER:sa}
spring.datasource.password=${DB_PASSWORD:}
spring.datasource.driver-class-name=org.h2.Driver

spring.sql.init.mode=always
spring.sql.init.schema-locations=classpath:schema.sql
spring.sql.init.data-locations=classpath:data.sql

spring.h2.console.enabled=true          # /h2-console — false
en PROD
spring.h2.console.path=/h2-console
```

URL	Dónde viven los datos
`jdbc:h2:mem:funkos`	En memoria: se borran al parar
`jdbc:h2:file:./datos/funkos`	**En fichero: sobreviven**

AVISO `AUTO_SERVER=TRUE` permite que la consola de H2 y tu app abran la base a la vez. Sin él: `Database may be already in use`. **AVISO** Con fichero, `schema.sql` se ejecuta **en cada arranque** → `CREATE TABLE IF NOT EXISTS` y `MERGE … KEY(id)` en vez de `INSERT`. **AVISO** Si insertas con ids fijos: `ALTER TABLE t ALTER COLUMN id RESTART WITH n+1`, o el primer POST choca.

```
@Component
public class FunkoRowMapper implements RowMapper<Funko> {
    @Override public Funko mapRow(ResultSet rs, int n) throws
SQLException {
        return Funko.builder()
                .id(rs.getLong("id"))
                .nombre(rs.getString("nombre"))
                .precio(rs.getBigDecimal("precio"))
                .createdAt(rs.getTimestamp("created_at").toLocalI
                .build();
    }
}
```

```
@Repository
public class R {
    private final JdbcClient jdbc;
    public R(JdbcClient jdbc) { this.jdbc = jdbc; }

    // LISTA
    jdbc.sql("SELECT * FROM funkos ORDER BY
id").query(rowMapper).list();

    // UNO: devuelve Optional
    jdbc.sql("SELECT * FROM funkos WHERE id = :id")
        .param("id", id).query(rowMapper).optional();

    // UN ESCALAR
    jdbc.sql("SELECT COUNT(*) FROM funkos WHERE nombre = :n")
        .param("n", nombre).query(Integer.class).single();

    // INSERT con id generado
    var keys = new GeneratedKeyHolder();
    jdbc.sql("INSERT INTO funkos (nombre, precio) VALUES
(:nombre, :precio)")
        .param("nombre", f.getNombre()).param("precio",
f.getPrecio())
        .update(keys);
    f.setId(((Number) keys.getKeys().get("id")).longValue());

    // UPDATE / DELETE: devuelven filas afectadas
    int filas = jdbc.sql("DELETE FROM funkos WHERE id
= :id").param("id", id).update();

    // VARIOS PARÁMETROS DE GOLPE

jdbc.sql(sql).params(mapaDeParametros).query(rowMapper).list();
}
```

Filtros opcionales y combinables:

```
var sql = new StringBuilder("SELECT * FROM funkos WHERE 1 =
1");
var params = new HashMap<String, Object>();

categoria.ifPresent(c -> {
    sql.append(" AND UPPER(categoria) =
UPPER(:categoria)");   // ✅ texto FIJO
    params.put("categoria",
c);                                  // ✅ valor parametrizado
});
return
jdbc.sql(sql.toString()).params(params).query(rowMapper).list();
```

AVISO NUNCA `sql.append(" AND categoria = '" + c + "'")` → inyección SQL. **AVISO** El `WHERE 1 = 1` evita llevar la cuenta de cuál es el primer filtro.

Comprobar y escribir en la MISMA sentencia (condición de carrera):

```
UPDATE productos SET stock = stock - :cantidad
 WHERE id = :id AND stock >= :cantidad
```

```
if (repositorio.descontarStock(id, cantidad) == 0)
    throw new StockInsuficienteException(...);      // afectó
a 0 filas → no había
```

AVISO «Leer, comprobar con un `if`, y después escribir» deja hueco para que otra petición se cuele en medio y el stock quede negativo.

	UT3, a mano	UT4, `JdbcClient`
Conexión	`DriverManager` + `try-with-resources`	**Spring**, de un pool
Parámetros	`ps.setString(1, …)` desde 1	`:nombre`
`ResultSet`	`while (rs.next())`	`RowMapper` + `.list()`
Cero o uno	`rs.next() ? … : null`	`.optional()`
Id generado	`RETURN_GENERATED_KEYS`	`GeneratedKeyHolder`
Cerrar	A mano	**Spring**

9 · `@Transactional`

```
@Transactional(readOnly = true)              // lecturas
@Transactional                               // escrituras
@Transactional(rollbackFor = Exception.class)
```

Va en el `@Service`, no en el repositorio. Una operación de negocio que toca varias tablas o filas tiene que ser atómica: o todo o nada.

AVISO Solo revierte con `RuntimeException`. Una comprobada **confirma** la transacción a medias. **AVISO** Si **capturas** la excepción dentro, **no hay rollback**: tiene que salir del método. **AVISO** Funciona **por proxy**: `this.metodo()` no abre transacción.

10 · Configuración

```
server.port=${PORT:8080}                    # variable, o 8080
si no existe
spring.profiles.active=${SPRING_PROFILES_ACTIVE:dev}
api.path=/api
api.version=v1
server.error.include-message=always         # never en prod
server.error.include-stacktrace=never       # SIEMPRE never en
prod
logging.level.es.iesx=DEBUG
```

Prioridad (gana el de arriba):

```
1. --server.port=9000          (línea de comandos)
2. SERVER_PORT=9000            (variable de entorno)
3. application-{perfil}.properties
4. application.properties
5. defectos de Spring Boot
```

`server.port` → `SERVER_PORT` · `spring.datasource.url` → `SPRING_DATASOURCE_URL`

```
@Value("${api.version:v1}") private String
version;              // un valor

@ConfigurationProperties(prefix =
"funkos")                       // un grupo
public record Props(Upload upload) {
    public record Upload(String directorio, long tamanoMaximo)
{}
}
// + @EnableConfigurationProperties(Props.class) en la clase
principal
```

```
@Bean @Profile("dev")
public CommandLineRunner datos(S servicio) { return args ->
{ … }; }
```

	dev	prod
`h2.console.enabled`	`true`	`false`
`include-stacktrace`	`on_param`	`never`
`logging.level.root`	`DEBUG`	`WARN`
URL de H2	fichero local	variable de entorno

`.env` → `.gitignore`. `.env.example` con las claves vacías → sí a Git. **IntelliJ:** *Edit Configurations → Environment variables*, o el plugin **EnvFile** apuntando al `.env`.

11 · Tests

```
// 1 · Servicio, sin Spring: milisegundos
@ExtendWith(MockitoExtension.class)
class STest {
    @Mock Repo repo;
    @Spy  Mapper mapper = new Mapper();
    @InjectMocks ServiceImpl servicio;

    @Test void noExisteLanza404() {
        when(repo.findById(99L)).thenReturn(Optional.empty());
        assertThrows(NotFoundException.class, () ->
servicio.findById(99L));
        verify(repo, times(1)).findById(99L);
        verify(repo, never()).save(any());
    }
}
```

```
// 2 · Capa web, con MockMvc: caza el @Valid olvidado
@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
class CTest {
    @Autowired MockMvc mockMvc;
    @MockitoBean Service servicio;          // NO @MockBean:
obsoleta desde 3.4

    @Test void postInvalidoDa400() throws Exception {
        mockMvc.perform(post("/api/v1/funkos")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content("""
                        {"nombre":"","precio":-1}"""))
                .andExpect(status().isBadRequest())
                .andExpect(jsonPath("$.campos.nombre").exists());
    }
}
```

```
status().isOk()  isCreated()  isNoContent()  isBadRequest()
isNotFound()  isConflict()
jsonPath("$.nombre").value("Casco")
jsonPath("$").isArray()   jsonPath("$", hasSize(3))
header().string("Location", "/api/v1/funkos/1")
```

```
// 3 · La transacción: comparar el estado antes y después de
un fallo
@Test void siFallaUnaLineaNoSeDescuentaNingunStock() {
    var antes = productos.findAll().stream()
            .collect(Collectors.toMap(Producto::getId,
Producto::getStock));
    assertThrows(StockInsuficienteException.class, () ->
pedidos.crear(peticionQueFalla));
    var despues = productos.findAll().stream()
            .collect(Collectors.toMap(Producto::getId,
Producto::getStock));
    assertEquals(antes, despues);
}
```

```
./mvnw test
./mvnw test -Dtest="FunkosServiceImplTest"
./mvnw test -Dtest="FunkosServiceImplTest#noExisteLanza404"
```

12 · `curl`

```
curl -i localhost:8080/api/v1/funkos
curl -s "localhost:8080/api/v1/funkos?
categoria=disney&precioMax=20" | jq
curl -i -X POST localhost:8080/api/v1/funkos \
     -H "Content-Type: application/json" \
     -d '{"nombre":"Casco","precio":35.00,"cantidad":
1,"categoria":"OTROS"}'
curl -i -X PATCH localhost:8080/api/v1/funkos/1 \
     -H "Content-Type: application/json" -d '{"precio":40.00}'
curl -i -X DELETE localhost:8080/api/v1/funkos/1
```

Los quince errores del examen

	Síntoma	Causa
1	500 en vez de 404	`findById(id).get()` sobre un `Optional` vacío
2	201 con datos basura	`@Valid` **olvidado**
3	`cantidad: -5` dentro de una lista pasa la validación	Falta el `@Valid` **anidado**
4	204 al borrar algo que no existe	El `delete` no mira las filas afectadas
5	415 en el POST	Falta `Content-Type: application/json`
6	`@Cacheable` no cachea	Falta `@EnableCaching`
7	Datos viejos en el listado	Falta `@CacheEvict(allEntries = true)`
8	Bean no encontrado	Clase fuera del paquete escaneado, o sin `@Service`
9	PATCH que borra campos	PUT y PATCH usando el mismo método del mapeador
10	Contador compartido entre usuarios	Estado mutable en un bean singleton
11	`Table "X" already exists` al segundo arranque	Falta `CREATE TABLE IF NOT EXISTS`
12	El primer POST choca con un id existente	Falta `ALTER TABLE … RESTART WITH n`
13	`Database may be already in use`	Falta `AUTO_SERVER=TRUE` en la URL de H2
14	Stock negativo con dos peticiones a la vez	«Leer, comprobar, escribir» en vez de `UPDATE … WHERE stock >= :n`
15	Stock descontado sin pedido creado	Falta `@Transactional`, o la excepción se capturó dentro